

2.3 Quicksort

Dateiname: quicksort.s

```
.data
lo:
    .word 0xff
    .word 0x17
    .word 0x38
    .word 0x25
    .word 0xa3
    .word 0xc8
    .word 0x84
    .word 0x91
    .word 0x47
    .word 0xb3
    .word 0x91
    .word 0x71
    .word 0xb4
    .word 0xf3
    .word 0x00
hi:
    .word 0x05

.text
j main

swap:
#####
# Swap words [a] with [b]
#
# IN
# a0 = a
# a1 = b
#####
# load values to be exchanged
        lw t0 0 a0
        lw t1 0 a1
# write
        sw t0 0 a1
        sw t1 0 a0
        ret

partition:
#####
# Partition array around pivot using Hoare's method
# pivot = first element
#
# IN
# a0 = start address
# a1 = end address
#
# OUT
# a0 = pivot address
#
# REGISTER MAP
# s0 = i (addr)
# s1 = j (addr)
# s2 = [i]
# s3 = [j]
# s4 = pivot value
# s5 = start address of array (copy of a0)
# s6 = end address of array (copy of a1)
#####
# save return address
        addi sp sp -16
        sw ra 12 sp
```

```

# initialize registers
    addi s0 a0 -4
    addi s1 a1 4
    lw s4 0 a0
    mv s5 a0
    mv s6 a1

loop:
    # scan forward until element >= pivot
    inc_i:
    beq s0 s6 skip_i
    addi s0 s0 4
    lw s2 0 s0
    blt s2 s4 inc_i
    skip_i:
    # state: [i] >= pivot

    # scan backward until element <= pivot
    dec_j:
    beq s1 s5 skip_j
    addi s1 s1 -4
    lw s3 0 s1
    bgt s3 s4 dec_j
    skip_j:
    # state: [j] <= pivot

    # if i >= j return j
    bgeu s0 s1 done_partition

    # We know that [i] > [j]
    # swap(i, j)
    mv a0 s0
    mv a1 s1
    call swap

    j loop

done_partition:
    # restore return address and deallocate stack space
    lw ra 12 sp
    addi sp sp 16

    # return j
    mv a0 s1
    ret

quicksort:
#####
# Perform quicksort on array of word-sized integers
# MUTATES
#
# IN
# a0 = start address
# a1 = end address
#####
    # base case
    # if start address >= end address: return
    bgeu a0 a1 done_quicksort

    # Establish stack frame for recursive quicksort call
    # First, allocate space on stack aligned to 128 bits as per RISC-V ABI
    addi sp sp -16
    sw ra 12 sp
    sw a0 8 sp
    sw a1 4 sp

```

```

    call partition

    # State after returning from partition():
    # a0 = address of pivot (return value of partition())
    # Save this on stack because we are calling quicksort soon
    sw a0 0 sp

    # Sort high array (values greater than pivot)
    # -> quicksort(pivot+1, end)
    addi a0 a0 4
    lw a1 4 sp
    call quicksort

    # Sort low array (values smaller than pivot)
    # -> quicksort(start, pivot)
    lw a0 8 sp
    lw a1 0 sp
    call quicksort

    # Clean up
    lw ra 12 sp
    addi sp sp 16

done_quicksort:
    ret

main:
    # lo = start address of array
    # hi = end address of array
    la a0 lo
    la a1 hi
    call quicksort

```

2.4 Quicksort Stack Frame

Stackframe während der ersten Partition und des ersten Swaps:

quicksort (16 bytes)	
+12	return address -> main
+8	start address of arr (a0 arg)
+4	end address of arr (a1 arg)
partition (16 bytes)	
+12	return address -> quicksort
swap (0 bytes)	

+Offset aus der Sicht des Callees
↑ höhere Adressen ↓ niedrigere Adressen